

useCallback

```
const memoizedCallback = useCallback(
  () => {
    doSomething(value1, value2);
  },
  [value1, value2],
);
```

The next one is *useCallback*, which returns a memoized callback.

As you might know, memoization is an optimisation technique that is used primarily to speed up the process by storing the results of expensive calls and returning the cached results when the same inputs occur again. In the example above, we passed an inline callback and an array of dependencies. It will return a memoized version of the callback that changes only if one of the dependencies, in this case, value 1 and value 2, has changed.

This hook is useful when you want to prevent unnecessary re-renders when passing callbacks to optimise child components. Notice that all the values used in the callback are present in the dependency array. Sometimes with larger code, we can miss this, so we need to give particular attention to it. Also, there are some packages that can be used to throw warnings in case we miss them. But honestly, making it automatic would be the best solution that may be implemented in the future by React developers.

useMemo

```
const memoizedValue = useMemo(() => createValue(v1, v2),
[v1, v2]);
```

useMemo is basically used for optimising performance and returning a memoized value. It's similar to *useCallback*; we can have an array of dependencies and a create function. Here the create function will only run when the dependency is changed. If it hasn't changed, it will return the same value. So the use case for this would be in a scenario where you need to perform extensive calculations. By using *useMemo* you can ensure that time is saved when the variables involved have the same value.

But how is it different from *useEffect*? For the functions that we pass to *useMemo*, we can ensure that as they are executed during rendering, time is saved when the variables involved are the same. Also, you can see that *useEffect* is generally used for the side effects, which shouldn't run while the component is rendering. So that's the basic difference between *useMemo* and *useEffect* -- *useMemo* runs during rendering and *useEffect* does not.

We should be cautious when we use *useMemo*. There are instances wherein to utilise memory, React may forget and

re-calculate the values of future renders. In such cases, it is dangerous to totally depend on the execution of *useMemo*. As a result, the React team encourages us to ensure that the code runs properly without *useMemo* and is then added for performance optimisation.

useRef

Imagine an empty space. This space is in your component and we can put any value in it throughout the component's life cycle. *useRef* is basically that space and the value is the DOM node. As you know, primarily *refs* have been used to access the DOM; so if you pass a *ref* object to React (like with the input element in the code below), it will introduce a property called *current*, and whenever we access this property of the *ref* object, it will give us the DOM node. It's similar to having fields in classes. But if the *useRef* is essentially creating an object, why not do so explicitly? You'll realise that *useRef* gives us the same *ref* object on every render and that's what makes it useful. Also, you must keep in mind that *useRef* does not notify us when the content changes; it just provides us with the latest value whenever we access its current property.

```
const refContainer = useRef(initialValue);

function TextInputWithFocusButton() {
  const inputEl = useRef(null);
  const onClick = () => {
    // `current` points to the mounted text input element
    inputEl.current.focus();
  };
  return (
    <>
      <input ref={inputEl} type="text" />
      <button onClick={onClick}>Focus the input</
button>
    </>
  );
}
```

If you want to have any code when the *ref* is attached or detached, you should use the callback *refs*.

A few more hooks and custom hooks will be discussed in the next and final part of this article to complete our understanding of them. It would be a good idea to do a bit of self-study on custom hooks till then. **END** 🐼

By: Janhavi Dahihande

The author is a software engineer based in San Francisco. She is interested in learning and talking about new technologies, and likes contributing to open source. When not coding, she likes to attend book club events, travel, and try out new cuisines.